

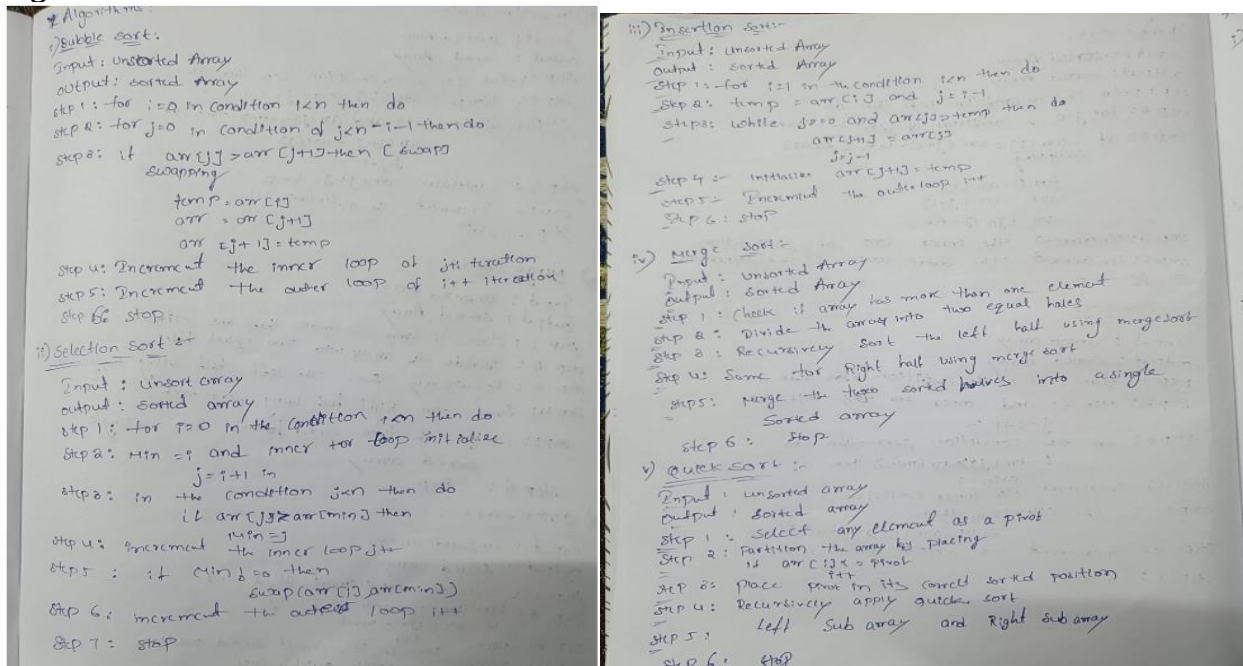
Practical: 1

29/07/2026

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Algorithm:



Program:

1. Bubble sort, selection sort, Insertion sort

```
#include <iostream>
using namespace std;
```

```
class classname {
public:
```

```
void Func() {
    int array1[100];
    int input;
```

```
    cout << "Enter the size of array: ";
    cin >> input;
```

```
    cout << "Enter the Elements in array:\n";
    for (int i = 0; i < input; i++) {
```

```
        cin >> array1[i];
    }

    cout << "\n";
    display(array1, input);

    cout << "\n\n";
    Bubble(array1, input);

    cout << "\n\n";
    Selection(array1, input);

    cout << "\n\n";
    Insertion(array1, input);
}

void display(int array1[], int n) {
    cout << "Array: ";
    for (int i = 0; i < n; i++) {
        cout << array1[i] << " ";
    }
}

// Bubble Sort
void Bubble(int array1[], int n) {
    cout << "Bubble Sort:\n";

    int arr[100];

    for (int i = 0; i < n; i++) {
        arr[i] = array1[i];
    }

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {

            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    display(arr, n);
}
```

```
}

// Selection Sort
void Selection(int array1[], int n) {
    cout << "Selection Sort:\n";

    int arr[100];

    for (int i = 0; i < n; i++) {
        arr[i] = array1[i];
    }

    for (int i = 0; i < n - 1; i++) {

        int min = i;

        for (int j = i + 1; j < n; j++) {

            if (arr[j] < arr[min]) {
                min = j;
            }
        }

        if (min != i) {
            int temp = arr[i];
            arr[i] = arr[min];
            arr[min] = temp;
        }
    }

    display(arr, n);
}

// Insertion Sort
void Insertion(int array1[], int n) {
    cout << "Insertion Sort:\n";

    int arr[100];

    for (int i = 0; i < n; i++) {
        arr[i] = array1[i];
    }

    for (int i = 1; i < n; i++) {

        int temp = arr[i];
```

```
int j = i - 1;

while (j >= 0 && arr[j] > temp) {
    arr[j + 1] = arr[j];
    j--;
}

arr[j + 1] = temp;
}

display(arr, n);
};
```

```
int main() {

    classname obj;

    obj.Func();

    return 0;
}
```

2. Quick sort and merge sort:

```
#include <iostream>
using namespace std;
```

```
void display(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
// ----- QUICK SORT -----
```

```
int partition(int arr[], int low, int high)
{
    int pivot = arr[low];
    int start = low;
    int end = high;

    while (start < end)
    {
        while (arr[start] <= pivot && start < high)
```

```
{
    start++;
}

while (arr[end] > pivot)
{
    end--;
}

if (start < end)
{
    swap(arr[start], arr[end]);
}
}

swap(arr[low], arr[end]);

return end;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int loc = partition(arr, low, high);

        quickSort(arr, low, loc - 1);
        quickSort(arr, loc + 1, high);
    }
}

// ----- MERGE SORT -----

void merge(int arr[], int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = 0;

    int temp[100];

    while (i <= mid && j <= high)
    {
        if (arr[i] <= arr[j])
        {
            temp[k] = arr[i];
            i++;
        }
    }
}
```

```
    }
    else
    {
        temp[k] = arr[j];
        j++;
    }
    k++;
}

while (i <= mid)
{
    temp[k] = arr[i];
    i++;
    k++;
}

while (j <= high)
{
    temp[k] = arr[j];
    j++;
    k++;
}

for (i = low, k = 0; i <= high; i++, k++)
{
    arr[i] = temp[k];
}
}

void mergeSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);

        merge(arr, low, mid, high);
    }
}

// ----- MAIN FUNCTION -----

int main()
{
    int arr[100];
```

```
int n;

cout << "Enter the size of array: ";
cin >> n;

cout << "Enter the elements of array:" << endl;

for (int i = 0; i < n; i++)
{
    cin >> arr[i];
}

cout << endl;
cout << "Original array: ";
display(arr, n);

// Quick Sort
quickSort(arr, 0, n - 1);

cout << endl;
cout << "Quicksort sorted array: ";
display(arr, n);

// Merge Sort
mergeSort(arr, 0, n - 1);

cout << endl;
cout << "Mergesort sorted array: ";
display(arr, n);

return 0;

}
```

Output:1.

```
Enter the size of array: 6
Enter the Elements in array:
3 4 2 5 1 6
```

```
Array: 3 4 2 5 1 6
```

```
Bubble Sort:
```

```
Array: 1 2 3 4 5 6
```

```
Selection Sort:
```

```
Array: 1 2 3 4 5 6
```

```
Insertion Sort:
```

```
Array: 1 2 3 4 5 6
```

2.

```
Enter the size of array: 5
Enter the elements of array:
21 23 14 5 24
```

```
Original array: 21 23 14 5 24
```

```
Quicksort sorted array: 5 14 21 23 24
```

```
Mergesort sorted array: 5 14 21 23 24
```

```
=== Code Execution Successful ===
```


Time complexity:

Bubble Sort:

```

for (i = 0; i < n-1; i++) {
    for (j = i+1; j < n; j++) {
        if (arr[i] > arr[j]) {
            swap(arr[i], arr[j]);
        }
    }
}

```

Total $T(n) = (n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2}$
 $= O(n^2)$

Selection Sort:

```

for (i = 0; i < n-1; i++) {
    min = i;
    for (j = i+1; j < n; j++) {
        if (arr[j] < arr[min]) {
            min = j;
        }
    }
    swap(arr[i], arr[min]);
}

```

Total $T(n) = (n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2}$
 $= O(n^2)$

Insertion Sort:

```

for (i = 1; i < n; i++) {
    j = i-1;
    while (j > 0 & arr[j] > arr[j+1]) {
        swap(arr[j], arr[j+1]);
        j--;
    }
}

```

Total $T(n) = 1 + 2 + \dots + (n-1)$
 $= \frac{n(n-1)}{2}$
 $= O(n^2)$

Merge Sort:

```

mergeSort(arr, left, right) {
    if (left < right) {
        mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid+1, right);
        merge(arr, left, mid, right);
    }
}

```

$T(n) = 2T(n/2) + O(n)$
 $T(n) = O(n \log n)$

Quick Sort:

```

quickSort(arr, s, e) {
    if (s < e) {
        p = partition(arr, s, e);
        quickSort(arr, s, p-1);
        quickSort(arr, p+1, e);
    }
}

```

$T(n) = O(n) \times O(\log n)$
 $T(n) = O(n \log n)$
 $T(n) = T(n-1) + n$
 $T(n) = T(n-2) + 2n$
 $T(n) = T(n-3) + 3n$
 $T(n) = O(n^2)$

Conclusion:

The implementation and time analysis of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort helped in understanding the performance and working principles of different sorting algorithms. Bubble Sort, Selection Sort, and Insertion Sort are simple and easy to implement but generally require $O(n^2)$ time in the average and worst cases. Merge Sort provides better performance with $O(n \log n)$ time complexity and is suitable for large datasets, although it requires additional memory. Quick Sort also has an average time complexity of $O(n \log n)$ and is generally very efficient in practice, but its worst-case complexity can be $O(n^2)$ depending on pivot selection.